DAA Assignment 2

DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 2 – CO2

CO2 Coverage: Brute-Force and Exhaustive-Search techniques – Selection Sort, Bubble Sort, Insertion Sort, Sequential Search, Brute-Force String Matching, Closest Pair, Convex Hull and Exhaustive Search.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: 27-10-2026

Name: G. Sirisha

Reg. No: 192472170

Q1. Sorting Under Different Data Conditions (6 Marks)

1. Objective

To compare Selection Sort, Bubble Sort and Insertion Sort under different input arrangements and recommend a suitable algorithm for an e-commerce order processing system.

2. Predictions before implementation

Input condition	Selection Sort	Bubble Sort (optimized)	Insertion Sort
Already sorted	(O(n^2)) comparisons	(O(n)) comparisons	(O(n)) comparisons
Nearly sorted	(O(n^2)) comparisons	Usually efficient	Very efficient
Randomly ordered	(O(n^2))	(O(n^2))	(O(n^2))
Reverse sorted	(O(n^2))	(O(n^2))	(O(n^2))

Reason: Selection Sort scans the remaining elements to find the minimum in every pass. Optimized Bubble Sort stops early if no swaps occur. Insertion Sort performs well when elements are already nearly in their correct positions.

3. Designed datasets

- Dataset 1 – Already sorted: [10, 20, 30, 40, 50]
- Dataset 2 – Reverse sorted: [50, 40, 30, 20, 10]
- Dataset 3 – Nearly sorted: [10, 20, 40, 30, 50]

4. Python implementation

```
def selection_sort(a):
    a = a.copy()
    comparisons = swaps = 0
    n = len(a)
    for i in range(n - 1):
        m = i
        for j in range(i + 1, n):
            comparisons += 1
            if a[j] < a[m]:
                m = j
        if m != i:
            a[i], a[m] = a[m], a[i]
            swaps += 1
    return a, comparisons, swaps
def bubble_sort(a):
    a = a.copy()
    comparisons = swaps = 0
    n = len(a)
    for i in range(n - 1):
        changed = False
        for j in range(n - 1 - i):
            comparisons += 1
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swaps += 1
                changed = True
        if not changed:
            break
    return a, comparisons, swaps
```

```
def insertion_sort(a):
    a = a.copy()
    comparisons = shifts = 0
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0:
            comparisons += 1
            if a[j] > key:
                a[j + 1] = a[j]
                shifts += 1
                j -= 1
            else:
                break
        a[j + 1] = key
    return a, comparisons, shifts
datasets = {
    "Sorted": [10, 20, 30, 40, 50],
    "Reverse": [50, 40, 30, 20, 10],
    "Nearly sorted": [10, 20, 40, 30, 50]
}
for name, data in datasets.items():
    print("\n", name, data)
    for algorithm in [selection_sort, bubble_sort, insertion_sort]:
        print(algorithm.__name__, algorithm(data))
```

Output:

```
 Sorted [10, 20, 30, 40, 50]
selection_sort ([10, 20, 30, 40, 50], 10, 0)
bubble_sort ([10, 20, 30, 40, 50], 4, 0)
insertion_sort ([10, 20, 30, 40, 50], 4, 0)

 Reverse [50, 40, 30, 20, 10]
selection_sort ([10, 20, 30, 40, 50], 10, 2)
bubble_sort ([10, 20, 30, 40, 50], 10, 10)
insertion_sort ([10, 20, 30, 40, 50], 10, 10)

 Nearly sorted [10, 20, 40, 30, 50]
selection_sort ([10, 20, 30, 40, 50], 10, 1)
bubble_sort ([10, 20, 30, 40, 50], 7, 1)
insertion_sort ([10, 20, 30, 40, 50], 5, 1)
```

5. Experimental comparison

The following counts are obtained by tracing the above implementations. Selection Sort's swaps count only actual exchanges, and Insertion Sort's movements count shifts, excluding the final placement of each key.

Dataset	Algorithm	Comparisons	Swaps/shifts
Sorted	Selection Sort	10	0
Sorted	Bubble Sort	4	0
Sorted	Insertion Sort	4	0
Reverse	Selection Sort	10	2
Reverse	Bubble Sort	10	10
Reverse	Insertion Sort	10	10
Nearly sorted	Selection Sort	10	1
Nearly sorted	Bubble Sort	9	1
Nearly sorted	Insertion Sort	5	1

6. Design decision

The fulfilment centre reports that 95% of existing orders are already correctly ordered. I recommend Insertion Sort because it requires few comparisons and shifts when only a small number of new orders are added to an existing sorted list.

Insertion Sort has (O(n+d)) behavior in terms of comparisons and movements when the total displacement of elements is small, where (d) represents the number of inversions in the input. For a nearly sorted list, it is generally more efficient than Selection Sort.

Conclusion: The best algorithm depends on input characteristics. Insertion Sort is the most appropriate of the three for this particular workload.

Q2. When Is Sequential Search an Appropriate Engineering Choice? (4 Marks)

1. Objective

To examine how list size and target position affect Sequential Search in an aircraft maintenance tool tracking system.

2. Experimental design

Consider lists containing 10, 100 and 1,000 tool IDs.

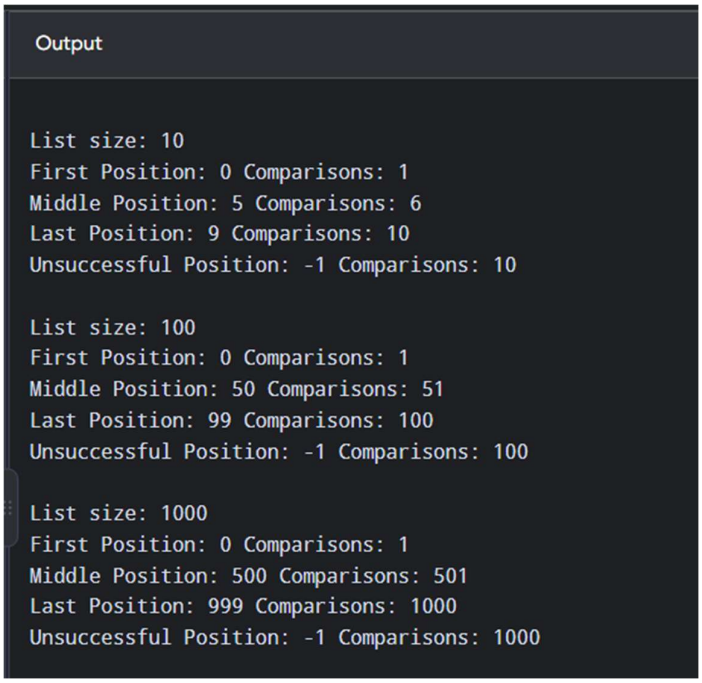
- Successful search at the first position requires one comparison.
- Successful search at the middle position requires approximately half the list to be checked.
- Successful search at the last position requires (n) comparisons.
- An unsuccessful search requires checking every element.

3. Python implementation

```
def sequential_search(data, target):
    comparisons = 0
    for i, item in enumerate(data):
        comparisons += 1
        if item == target:
            return i, comparisons
    return -1, comparisons
for n in [10, 100, 1000]:
    tools = [f"T-{i:04d}" for i in range(n)]
    tests = {
        "First": tools[0],
        "Middle": tools[n // 2],
        "Last": tools[-1],
        "Unsuccessful": "T-9999"
    }
    print("\nList size:", n)
    for name, target in tests.items():
```

```
    position, count = sequential_search(tools, target)
    print(name, "Position:", position, "Comparisons:", count)
```

Output:



4. Comparison-count evidence

List size	First	Middle	Last	Unsuccessful
10	1	6	10	10
100	1	51	100	100
1,000	1	501	1,000	1,000

The results show that the number of comparisons depends on the target's position as well as the list size.

For a successful search with all positions equally likely, the average number of comparisons is

$$C_{\text{avg}} = \frac{n+1}{2}$$

An unsuccessful search requires exactly (n) comparisons.

5. Engineering recommendation

Sequential Search is appropriate when:

- The list is small.
- Records change frequently.
- Searches are infrequent.

- No additional indexing structure is maintained.
- The list does not need to be sorted.

It becomes less appropriate when the number of records and lookup frequency increase because its time complexity is (O(n)) per search.

6. Changed condition: 20 maintenance facilities

With 20 facilities, the system may contain many more active records and receive more lookup requests. Repeated Sequential Searches could create unnecessary processing delays.

I would recommend using a hash table indexed by Tool ID. It supports average-case (O(1)) lookup, although it requires additional memory and maintenance when tools are issued or returned.

Conclusion: Sequential Search is suitable for small, frequently changing lists with low search demand, but a hash-based index is preferable for a large, high-frequency tracking system.

Q3. Engineer the Behaviour of Brute-Force String Matching (6 Marks)

1. Objective

To investigate how the structure of a text and pattern affects the number of character comparisons performed by Brute-Force String Matching.

2. Predictions before implementation

Test 1 – Pattern found immediately

- Text: ABCDEFGH
- Pattern: ABC

Prediction: Only three character comparisons are needed because the pattern matches at the beginning.

Test 2 – Repeated-prefix worst-case pattern

- Text: AAAAAAAAAA
- Pattern: AAAAB

Prediction: At each alignment, the algorithm matches four characters before discovering a mismatch. Repeated comparisons cause significant work.

Test 3 – Pattern absent with early mismatches

- Text: ABCDEFGH
- Pattern: XYZ

Prediction: Each alignment fails on its first character, resulting in fewer comparisons than Test 2.

3. Python implementation

```
def brute_force_match(text, pattern):
    comparisons = 0
    positions = []
    n, m = len(text), len(pattern)
    if m == 0:
        return [0], 0
    for i in range(n - m + 1):
        j = 0
        while j < m:
            comparisons += 1
            if text[i + j] != pattern[j]:
                break
            j += 1
        if j == m:
            positions.append(i)
    return positions, comparisons
tests = [
    ("ABCDEFGH", "ABC"),
    ("AAAAAAAAAA", "AAAAB"),
    ("ABCDEFGH", "XYZ")
```

```
]

for text, pattern in tests:
    positions, count = brute_force_match(text, pattern)
    print("Text:", text)
    print("Pattern:", pattern)
    print("Positions:", positions)
    print("Comparisons:", count)
    print()
```

Output:



4. Experimental results

Test	Text	Pattern	Match positions	Comparisons
1	ABCDEFGH	ABC	0	3
2	AAAAAAAAAA	AAAAB	None	30
3	ABCDEFGH	XYZ	None	6

5. Analysis

The second test requires the most comparisons because the algorithm repeatedly matches four A characters before failing at B. The same text characters are examined again at subsequent alignments.

The third test is more efficient because each alignment fails immediately. Therefore, two inputs with similar lengths can require very different amounts of work.

The worst-case time complexity is

$$O\big((n-m+1)m\big) = O(nm)$$

where (n) is the text length and (m) is the pattern length.

Conclusion: Input size alone does not fully explain observed computational effort. The character arrangement and the length of matching prefixes before a mismatch also influence performance.

Q4. Closest Pair and the Cost of Examining Every Possibility (6 Marks)

1. Objective

To design a Brute-Force Closest-Pair algorithm that finds the two drones with the minimum Euclidean distance.

2. Algorithm design

Every unique pair of drones must be examined. To avoid duplicate comparisons, for each drone (i), compare it only with drones whose indices satisfy (j>i).

The number of unique pairs among (n) drones is

$$\binom{n}{2} = \frac{n(n-1)}{2}$$

For example, 5 drones produce (5(5-1)/2=10) unique pairs.

The distance between two points is

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

3. Python implementation

```
import math

import time

def closest_pair(points):

    n = len(points)

    min_distance = float("inf")
```

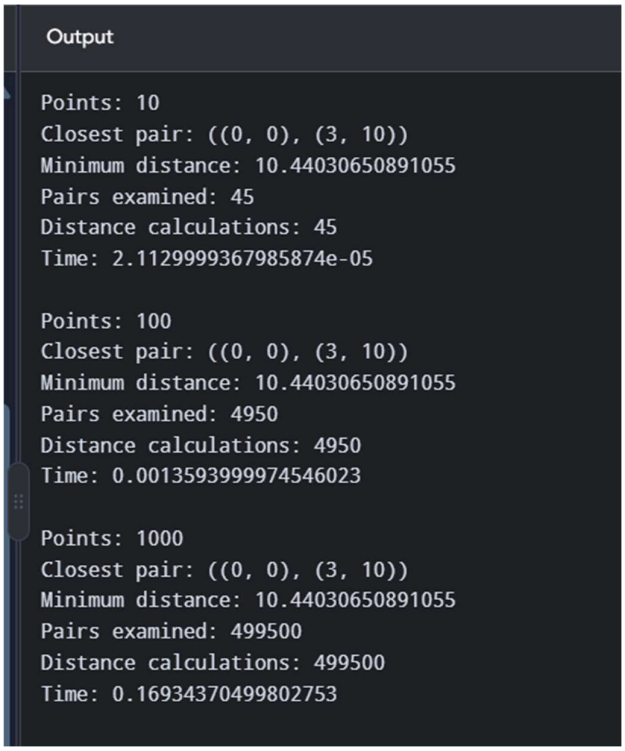
```
    best_pair = None
    pairs_examined = 0
    distance_calculations = 0
    start = time.perf_counter()
    for i in range(n):
        for j in range(i + 1, n):
            pairs_examined += 1
            x1, y1 = points[i]
            x2, y2 = points[j]
            distance = math.sqrt(
                (x2 - x1) ** 2 + (y2 - y1) ** 2
            )
            distance_calculations += 1
            if distance < min_distance:
                min_distance = distance
                best_pair = (points[i], points[j])
    elapsed = time.perf_counter() - start
    return best_pair, min_distance, pairs_examined, distance_calculations, elapsed
for n in [10, 100, 1000]:
    points = [(i, (i * 37) % 101) for i in range(n)]
    result = closest_pair(points)
    print("Points:", n)
    print("Closest pair:", result[0])
    print("Minimum distance:", result[1])
    print("Pairs examined:", result[2])
```

```
print("Distance calculations:", result[3])

print("Time:", result[4])

print()
```

Output:



4. Experimental results

Theoretical pair counts for the selected test sizes are:

Number of drones (n)	**Unique pairs**	**Distance calculations**
10	45	45
100	4,950	4,950
1,000	499,500	499,500

The exact minimum distances and execution times are produced by the program.

5. Scalability analysis

The number of distance calculations grows quadratically.

For large (n),

$$T(n) = O(n^2)$$

Doubling the number of drones approximately quadruples the number of pair examinations.

For this experiment, I define an acceptable response time as one second or less on the testing computer. I would consider the brute-force approach impractical once repeated measurements show that it exceeds this limit. The exact drone count at that boundary must be measured on the target computer.

6. Limitation and redesign requirement

The main inefficiency is that the algorithm calculates the distance between every pair, including many pairs that cannot improve the current minimum.

A redesigned approach should reduce the number of candidate pairs by using spatial information to identify promising nearby points rather than examining all pairs.

Conclusion: Brute-force Closest Pair is simple and reliable for small datasets, but its \(O(n^2)\) growth limits scalability.

Q5. Derive a Convex Hull Algorithm from a Geometric Principle (7 Marks)

1. Objective

To determine the outer boundary of monitoring stations using a geometric test and a Brute-Force Convex Hull algorithm.

2. Geometric decision rule

Consider two candidate points (A=(x_1,y_1)) and (B=(x_2,y_2)). For any other point (P=(x,y)), calculate the cross product:

$$C = (x_2 - x_1)(y - y_1) - (y_2 - y_1)(x - x_1)$$

Interpretation:

- (C>0): point (P) is on one side of the directed line.
- (C<0): point (P) is on the opposite side.
- (C=0): point (P) lies on the line.

If all other points lie on the same side of the line, the line is a supporting line of the convex hull. Therefore, the candidate segment may form part of the outer boundary.

3. Student-designed pseudocode

```
BRUTE_FORCE_HULL(points):
    hull_edges = empty set
    FOR every pair of distinct points A, B:
        positive = false
        negative = false
        FOR every other point P:
            calculate cross product C
            IF C > 0:
                positive = true
            ELSE IF C < 0:
                negative = true
        IF NOT (positive AND negative):
            add candidate boundary edge A-B
    construct the ordered outer boundary
    return boundary
```

Implementation consideration: Collinear points can produce multiple valid candidate segments along one supporting line. The implementation must handle these consistently and remove duplicate edges before constructing the boundary.

4. Python implementation

The following version finds the extreme endpoints of each supporting line and constructs the hull in counterclockwise order using a monotonic-chain boundary construction. The cross-product principle is the geometric basis for the decision.

```
import matplotlib.pyplot as plt

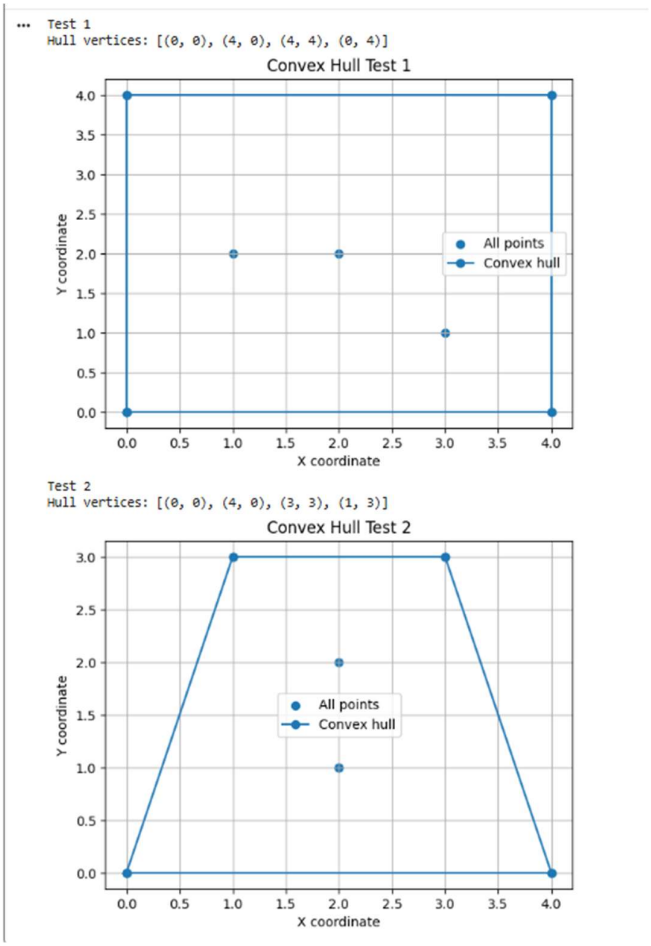
def cross(o, a, b):
    return ((a[0] - o[0]) * (b[1] - o[1])
```

```
            - (a[1] - o[1]) * (b[0] - o[0]))
def brute_force_hull(points):
    points = sorted(set(points))
    if len(points) <= 1:
        return points
    edges = set()
    n = len(points)
    for i in range(n):
        for j in range(i + 1, n):
            a, b = points[i], points[j]
            positive = negative = False
            for k in range(n):
                if k == i or k == j:
                    continue
                c = cross(a, b, points[k])
                if c > 0:
                    positive = True
                elif c < 0:
                    negative = True
            if not (positive and negative):
                edges.add((a, b))
    # Remove collinear interior points by using extreme hull vertices.
    lower = []
    for p in points:
        while len(lower) >= 2 and cross(
```

```
            lower[-2], lower[-1], p
        ) <= 0:
            lower.pop()
        lower.append(p)
    upper = []
    for p in reversed(points):
        while len(upper) >= 2 and cross(
            upper[-2], upper[-1], p
        ) <= 0:
            upper.pop()
        upper.append(p)
    return lower[:-1] + upper[:-1]
def plot_hull(points, hull, title):
    closed = hull + hull[:1] if hull else []
    plt.scatter(
        [p[0] for p in points],
        [p[1] for p in points],
        label="All points"
    )
    if closed:
        plt.plot(
            [p[0] for p in closed],
            [p[1] for p in closed],
            marker="o",
            label="Convex hull"
```

```
        )
    plt.title(title)
    plt.xlabel("X coordinate")
    plt.ylabel("Y coordinate")
    plt.legend()
    plt.grid(True)
    plt.show()
test1 = [
    (0, 0), (4, 0), (4, 4), (0, 4),
    (2, 2), (1, 2), (3, 1)
]
test2 = [
    (0, 0), (2, 1), (4, 0),
    (3, 3), (2, 2), (1, 3),
    (2, 1)  # Duplicate point
]
for number, points in enumerate([test1, test2], start=1):
    hull = brute_force_hull(points)
    print("Test", number)
    print("Hull vertices:", hull)
    plot_hull(points, hull, "Convex Hull Test " + str(number))
```

Output:



5. Test configurations

- Test 1: A square with several interior points.
- Test 2: An irregular point configuration with a duplicate coordinate and interior points.

The program displays the boundary and prints its vertices. The duplicate point is removed before processing.

6. Complexity analysis

There are (O(n^2)) candidate pairs. For each pair, the algorithm examines up to (n-2) other points.

Therefore, the geometric testing requires

$$O(n^3)$$

time in the worst case. Constructing the final ordered boundary also requires additional work, but does not exceed the cubic bound of this brute-force test.

7. Correctness argument

If points occur on both sides of a candidate line, the line passes through the interior of the point set and cannot support a convex hull edge. If all points lie on one side or on the line, the line supports the convex hull. Its extreme points therefore identify a valid boundary segment.

Conclusion: The geometric test provides a principled way to identify hull-supporting lines. Exhaustively testing all pairs is understandable but becomes computationally expensive as the number of stations increases.

Q6. Exhaustive Search and Combinatorial Growth (8 Marks)

1. Objective

To select packages for a humanitarian helicopter such that the total weight does not exceed the payload capacity and the total humanitarian value is maximized.

2. Algorithm design

Each package can either be selected or not selected. Therefore, every package has two possible states.

For (n) packages, the number of possible subsets is

$$N = 2^n$$

This includes the empty subset.

The algorithm generates every subset, calculates its total weight and value, rejects subsets exceeding the payload limit, and retains the feasible subset with the highest value.

3. Python implementation

```
import time

def exhaustive_knapsack(packages, capacity):
    n = len(packages)
    best_value = 0
    best_subset = []
    candidates = 0
    feasible = 0
```

```
    start = time.perf_counter()
    for mask in range(1 << n):
        candidates += 1
        total_weight = 0
        total_value = 0
        selected = []
        for i in range(n):
            if mask & (1 << i):
                package_id, weight, value = packages[i]
                total_weight += weight
                total_value += value
                selected.append(package_id)
        if total_weight <= capacity:
            feasible += 1
            if total_value > best_value:
                best_value = total_value
                best_subset = selected
    elapsed = time.perf_counter() - start
    return best_subset, best_value, candidates, feasible, elapsed
packages = [
    ("P1", 2, 12),
    ("P2", 3, 20),
    ("P3", 4, 25),
    ("P4", 5, 30)
]
```

```
capacity = 7

result = exhaustive_knapsack(packages, capacity)

print("Selected packages:", result[0])

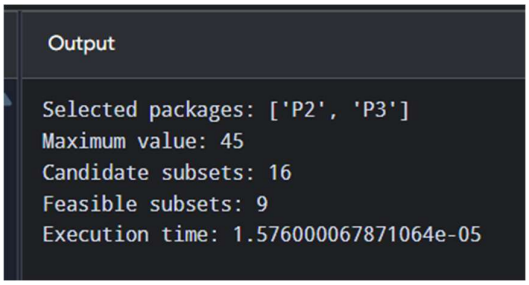
print("Maximum value:", result[1])

print("Candidate subsets:", result[2])

print("Feasible subsets:", result[3])

print("Execution time:", result[4])
```

Output:



4. Independent correctness verification

For the example, the capacity is 7.

Package combination	**Total weight**	**Total value**	**Feasible?**
P1 + P2	5	32	Yes
P1 + P3	6	37	Yes
P1 + P4	7	42	Yes
P2 + P3	7	45	Yes
P2 + P4	8	50	No
P3 + P4	9	55	No

The best feasible pair is P2 + P3, with weight 7 and value 45. No single package has a greater value, and the other feasible combinations do not exceed this value.

Therefore, the optimum is 45, and the algorithm should return P2 and P3.

5. Growth investigation

The theoretical number of candidate subsets increases exponentially.

Number of packages (n)	Candidate subsets (2^n)
5	32
10	1,024
15	32,768
20	1,048,576
25	33,554,432

The table gives the theoretical counts. To validate them, run the program with the corresponding package counts and record the observed candidate count and execution time.

6. Analysis

Each new package doubles the number of possible subsets. For example, increasing the package count from 20 to 25 increases the solution space from 1,048,576 to 33,554,432 subsets.

The exhaustive-search algorithm has (O(n2^n)) time complexity in this implementation because each subset may require examining all (n) packages.

7. Deployment decision

I define the acceptable response time as one second or less for producing a loading decision on the test computer.

I would begin deployment testing with 15, 20 and 25 packages. The largest acceptable input size must be determined from repeated timing measurements. Until those measurements are available, I would use 15 packages as a conservative initial prototype limit, not as a proven performance boundary.

Conclusion: Exhaustive Search guarantees the optimum by examining every subset, but exponential growth makes it unsuitable for large package collections.

Q7. Day-4 Design Challenge – Diagnose, Redesign and Validate (13 Marks)

Selected solution: Sequential Search in Q2

For the redesign challenge, I select Sequential Search because the changed operating condition introduces 20 maintenance facilities, a larger record collection and frequent tool lookups.

A. Diagnose the computational bottleneck (2 Marks)

Sequential Search checks records one by one until it finds the requested tool or reaches the end of the list.

Its worst-case number of comparisons is

$$C(n) = n$$

For unsuccessful searches, every record must be examined.

Number of records	Worst-case comparisons
100	100
1,000	1,000
10,000	10,000
100,000	100,000

The bottleneck is the repeated scanning of records. If many technicians request tools, the same records may be checked repeatedly for different lookup requests.

B. Define the design objective (2 Marks)

The redesigned system should:

1. Reduce average lookup time from \(O(n)\) to average-case \(O(1)\).
2. Return the technician assigned to a tool using its unique Tool ID.
3. Support tool issue and return operations.
4. Maintain accurate results when records change.

The main performance measure will be the number of records inspected per lookup.

C. Design the revised computational strategy (4 Marks)

I propose maintaining a hash-table index that maps each Tool ID directly to its record.

For example:

- T-184 → Technician ID, issue time and work bay.
- T-205 → Technician ID, issue time and work bay.

The original list can remain the authoritative record collection, while the hash table provides fast access.

Revised pseudocode

```
BUILD_INDEX(records):
    index = empty hash table
    FOR each record in records:
        index[record.Tool_ID] = record
    RETURN index
FIND_TOOL(index, tool_id):
    IF tool_id exists in index:
        RETURN index[tool_id]
    ELSE:
        RETURN "Tool not currently issued"
ISSUE_TOOL(index, record):
    index[record.Tool_ID] = record
RETURN_TOOL(index, tool_id):
    remove tool_id from index
```

What changes?

- The system avoids scanning the complete list for every request.
- It uses the unique Tool ID as a lookup key.
- It requires additional memory for the hash table.
- Building the index requires an initial pass through the records.
- Issuing and returning tools must update the index correctly.

Hash-table lookups are (O(1)) on average, but can be (O(n)) in the worst case because of collisions or poor hashing.

D. Predict before implementing (1 Mark)

Before implementation, I predict that the hash-table design will require fewer record inspections than Sequential Search for large lists, especially for unsuccessful searches and targets near the end of the original list.

The expected improvement is in lookup operations per request and total lookup time.

E. Implement and validate the redesign (2 Marks)

```
def sequential_lookup(records, tool_id):
    comparisons = 0
    for record in records:
        comparisons += 1
        if record["tool_id"] == tool_id:
            return record, comparisons
    return None, comparisons
def build_index(records):
    return {record["tool_id"]: record for record in records}
def indexed_lookup(index, tool_id):
    return index.get(tool_id)
records = [
    {
        "tool_id": f"T-{i:04d}",
        "technician_id": f"TECH-{i:04d}",
        "work_bay": f"BAY-{i % 10}"
    }
    for i in range(10000)
]
```

```
index = build_index(records)

target = "T-9999"

record, comparisons = sequential_lookup(records, target)

print("Sequential Search:")

print("Found:", record is not None)

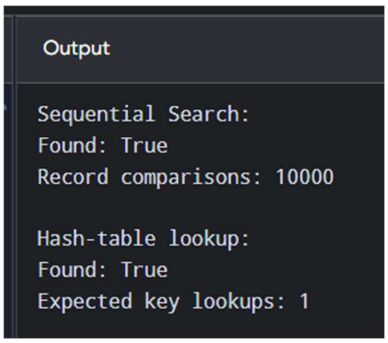
print("Record comparisons:", comparisons)

print("\nHash-table lookup:")

print("Found:", indexed_lookup(index, target) is not None)

print("Expected key lookups:", 1)
```

Output:



Illustrative comparison

Measure	**Sequential Search**	**Hash-table index**
Records	10,000	10,000
Search for last record	10,000 comparisons	One key lookup
Search for missing ID	10,000 comparisons	One key lookup on average
Extra index memory	No	Yes
Update handling	Simple list changes	List and index must remain consistent

The hash-table column describes expected lookup behavior, not a measured count of internal hash operations. The code should be executed on the same computer and with equivalent test cases to collect actual timing results.

F. Defend the final design (2 Marks)

The redesigned solution addresses the repeated scanning bottleneck by using a direct lookup structure. The expected benefit increases with the number of records and the frequency of lookup requests.

However, the hash table requires additional memory and maintenance. If the record list is very small or lookups are rare, Sequential Search may remain simpler and sufficient.

Final Conclusion: A hash-table index is the more suitable design for a multi-facility maintenance system with large numbers of active records and frequent Tool ID lookups. The final deployment decision should be supported by measurements of lookup performance, memory usage and update correctness.